

Day06 Part B 学习文档 v1.0 : Memory Architecture (记忆系统架构)

本文是《从零实现 Agent Runtime》学习阶段的 Day06 Part B 正式学习文档。

Part A 已经建立 Memory 的基础模型：Memory 不是聊天记录，也不是 Vector DB，而是 Runtime 从历史交互中抽取、管理、检索并投影给 LLM 的长期 State。Part B 进一步回答：

一条 Memory 从被保存，到未来被 Agent 使用，中间到底经过哪些组件？

本节定位

Day06 Part B 关注 Memory System 的架构层。

如果 Part A 解决的是：

什么是 Memory？为什么 Agent 需要 Memory？

那么 Part B 解决的是：

Memory System 应该由哪些组件组成？Store、Embedding、Vector DB、Retriever、Ranker、Context Builder 各自负责什么？

本节核心结论是：

Memory Architecture 不是“把历史信息存到向量数据库再搜出来”。它是一套由写入链和读取链组成的长期信息管理系统：写入链负责抽取、分类、去重、更新和持久化；读取链负责召回、排序、预算控制和上下文投影。

目录

Memory Architecture 全景

Memory System 的两条链路

Memory Store 是什么

Memory Entity 应该存什么

为什么 Memory Store 不等于 Vector DB

Embedding、Vector Search 与 Vector DB

Retriever 与 Ranker

Hybrid Retrieval

Context Budget 与 Memory Projection

Memory 到 Context Builder

Memory vs RAG

Knowledge Base 与企业 RAG

Memory Update 与 Similarity 的关系

Part B 最终模型

本 Part 核心知识点

写书 TODO

写书素材

本 Part 核心认知升级

工业级实现

知识地图

面试视角

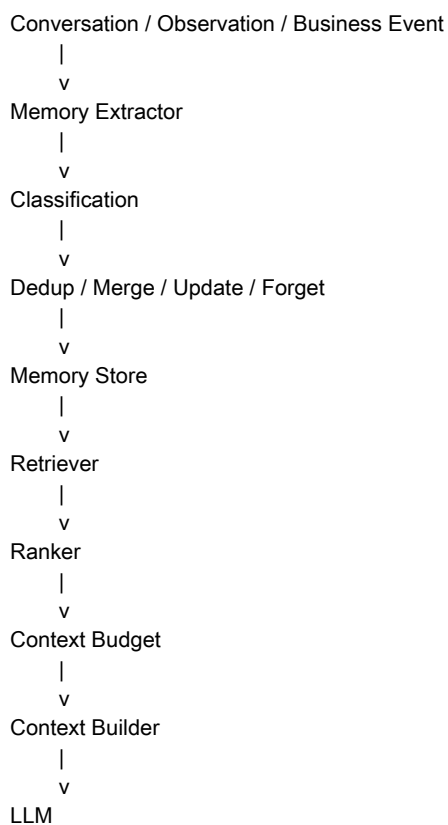
本章思考题

前置问题回收

下一节学习计划

Memory Architecture 全景

一个完整的 Memory System 大致可以被拆成：



这里每一层都不是装饰，而是各自承担不同的 Runtime 职责：

- Extractor 判断什么值得被记住
- Classification 判断 Memory 类型
- Store 负责长期持久化
- Retriever 从长期 Memory 空间里找候选
- Ranker 判断哪些候选更应该进入当前任务
- Context Budget 控制注入数量与 token 成本

- Context Builder 把 Memory 投影成 LLM 可读的上下文

所以，Memory Architecture 的核心不是数据库，而是信息从历史进入当前推理现场的完整路径。

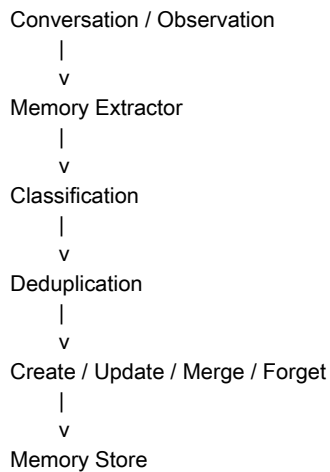
Memory System 的两条链路

Memory System 至少有两条不同链路：Write Path 和 Read Path。

写入链 Write Path

写入链解决的是：

什么东西应该被记住？

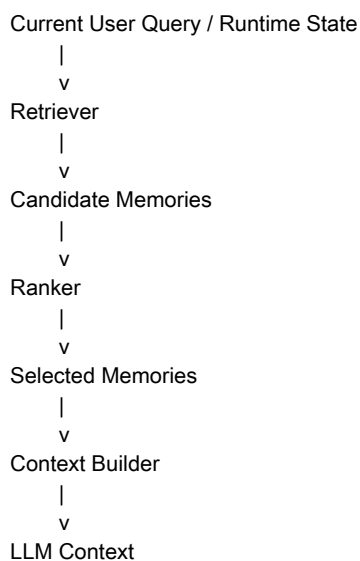


写入链关心的是长期信息质量。它不能把所有 Conversation 都保存成 Memory，否则长期记忆会被短期噪声污染。

读取链 Read Path

读取链解决的是：

当前这个任务应该想起什么？



很多简单 Memory Demo 只实现了 append，或者只实现了向量 top-k 搜索。但工业 Memory 同时需要写入链和读取链。没有写入质量，Memory 会脏；没有读取质量，Memory 会干扰当前任务。

Memory Store 是什么

Memory Store 是长期 Memory 的领域抽象。

它回答的问题是：

Memory 存在哪里？以什么结构保存？如何被读取、更新、删除和审计？

它不是简单的字符串列表，也不是单独的 Vector DB。更合理的 Memory Store 应该保存 Memory Entity。

例如用户说：

以后回答问题请多解释原理，不要只给答案。

Memory Store 不应该只存原句，而应该存结构化 Memory：

```
{
  "id": "mem_001",
  "user_id": "user_123",
  "type": "preference",
  "content": "User prefers detailed explanations instead of answer-only responses",
  "confidence": 0.92,
  "importance": 0.84,
  "source": "conversation",
  "created_at": "2026-08-12T09:00:00Z",
  "updated_at": "2026-08-12T09:00:00Z"
}
```

Memory Store 更像长期 State Repository，而不是“文本仓库”。

Memory Entity 应该存什么

一个 Memory Entity 通常至少需要这些字段：

- id：唯一标识
- user_id / agent_id / task_id：归属范围
- type：Memory 类型，例如 preference、profile、project、fact、skill
- content：长期信息内容
- embedding：用于语义检索的向量表示
- metadata：语言、领域、项目、来源、权限等结构化信息
- confidence：这条 Memory 的可信度
- importance：长期价值或优先级
- created_at / updated_at / last_accessed_at：生命周期时间信息
- status：active、deprecated、deleted 等状态

type 非常关键。因为同样一句话可能属于不同 Memory 语义：

User prefers TypeScript

可能是：

```
preference
technical_profile
project_context
```

如果没有类型，Runtime 很难判断这条 Memory 应该在什么场景下使用，也很难做 update、merge、forget。

为什么 Memory Store 不等于 Vector DB

Vector DB 是 Memory Store 的一种基础设施实现，但不是 Memory Store 本身。

Memory Store
≠
Vector Database

原因在于 Vector DB 主要解决：

Vector
|
v
Similarity Search

而 Memory Store 还必须解决：

- Memory 类型
- 用户归属
- 权限边界
- 来源记录
- 置信度
- 重要性
- 更新时间
- 冲突状态
- 生命周期
- 审计与删除

例如用户从“喜欢 Vue”变成“喜欢 React”，Memory Store 不能只新增一条向量相近的文本。它需要判断这是偏好变更、历史并存，还是项目上下文不同。

Old Memory:
User prefers Vue.

New Statement:
User now prefers React.

Possible Runtime Decision:
Update old memory
or
Create project-scoped memory
or
Mark old memory as deprecated

这类生命周期判断不是 Vector DB 自动完成的。

Embedding、Vector Search 与 Vector DB

Embedding 可以简单理解为：

把文本转换成能够表达语义关系的向量。

"User prefers TypeScript"
|
v
Embedding Model
|

v
[0.12, -0.44, 0.88, ...]

Vector Search 则是在向量空间里寻找语义接近的内容。

Query Vector
|
v
Compare with Memory Vectors
|
v
Top-K Similar Memories

Vector DB 负责存储和索引这些向量，并高效执行相似度检索。

三者职责不同：

Embedding Model
→ 把文本变成向量

Vector Search
→ 判断向量之间的相似度

Vector DB
→ 存储 / 索引 / 检索向量

但它们都不是完整的 Memory System。它们只是 Retrieval 的基础能力。

Retriever 与 Ranker

Retriever 和 Ranker 是两个不同组件。

Retriever → 找候选
Ranker → 排优先级

Retriever 负责从大规模 Memory Store 中召回一批可能相关的候选：

Memory Store
|
v
100,000 Memories
|
v
Retriever
|
v
100 Candidate Memories

Ranker 负责从候选里决定哪些更应该进入当前上下文：

100 Candidate Memories
|
v
Ranker
|
v
Top 5 Relevant Memories

相似度高不等于一定应该使用。Ranker 可能综合：

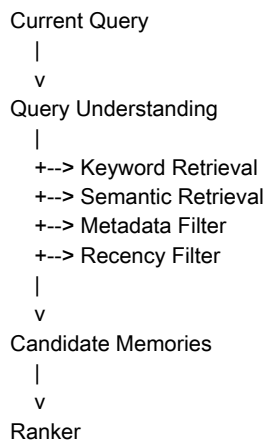
- semantic similarity
- memory type
- current task
- confidence

- importance
- recency
- user scope
- project scope
- safety policy
- context budget

例如当前用户正在写后端 Java 服务，“User prefers TypeScript”这条 Memory 语义上可能与编程相关，但对当前任务的重要性很低。它可以被 Retriever 找到，但 Ranker 应该降低它的优先级。

Hybrid Retrieval

工业 Memory Retrieval 通常不是单一 Vector Search，而是 Hybrid Retrieval。



Keyword Retrieval

Keyword Retrieval 适合精确匹配：

- 技术栈名称
- 项目名
- 文件名
- 用户明确说过的关键词
- 业务实体名

例如用户问“继续 Day06 Memory Architecture”，关键词检索可以直接命中 Day06、Memory、Architecture。

Semantic Retrieval

Semantic Retrieval 适合理解语义相近但字面不完全相同的表达。

Query:
"继续讲长期记忆怎么进入上下文"

Memory:
"User is learning Memory to Context Builder integration"

这类关系靠关键词可能漏掉，但向量相似度可以召回。

Metadata Filter

Metadata Filter 用来减少错误候选。

例如 Memory Store 中有多个用户：

```
user_001
user_002
user_003
```

当前请求来自 user_001，那么 Retriever 必须先按 user_id 过滤，否则就可能召回别人的 Memory。

这说明 Retrieval 不只是相似度问题，也是权限、范围和结构化约束问题。

Context Budget 与 Memory Projection

Retriever 找到的 Memory 不能直接全部塞给 LLM。

原因有三个：

Context Window 有限制

过多 Memory 会污染当前任务

Memory Store 的数据结构不是 LLM 最适合阅读的表达

所以 Memory Injection 本质上是一次 Projection：

```
Memory Store
|
v
Memory Records
|
v
Retrieval
|
v
Ranking
|
v
Selection under Context Budget
|
v
Memory Context Block
|
v
LLM
```

Memory Record 是给 Runtime 管理的，Memory Context Block 是给 LLM 阅读的。

例如 Runtime 内部 Memory Record 可能是：

```
{
  "id": "mem_001",
  "type": "preference",
  "content": "User prefers detailed explanations",
  "confidence": 0.92,
  "importance": 0.8,
  "created_at": "2026-08-12T09:00:00Z"
}
```

投影到 Prompt 里可能变成：

```
Relevant user preferences:
- The user prefers detailed explanations with underlying principles.
```

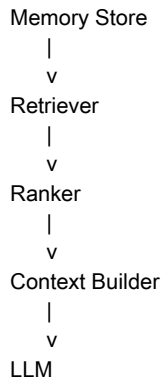
这就是 Context Builder 的职责：把 Runtime 内部状态编译成当前 LLM 可消费的上下文快照。

Memory 到 Context Builder

Day04 已经建立过一个关键认知：

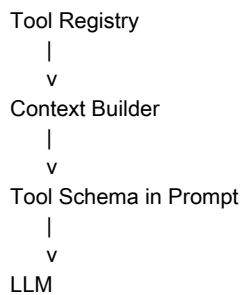
Context 不是所有状态，而是 Runtime 在某一时刻投影给 LLM 的 Snapshot。

Memory 也遵循同样原则。



LLM 不应该直接看到完整 Memory Store，而应该看到经过筛选、排序、预算控制和表达转换后的 Memory Context。

这和 Tool System 很像：



LLM 看到的不是 Runtime 内部 ToolRegistry 对象，而是 Tool Schema。同样，LLM 看到的也不是 Memory Store 对象，而是 Memory Projection。

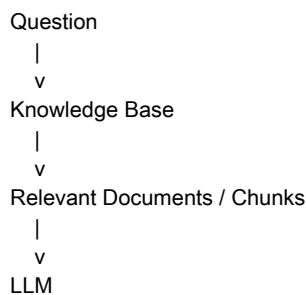
Memory vs RAG

Memory 和 RAG 可以共享底层 Retrieval 技术，但它们不是同一个概念。

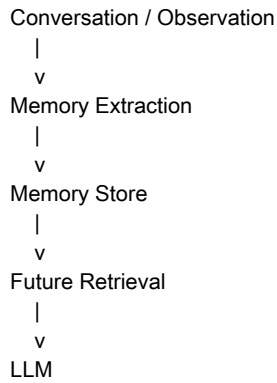
Memory
→ Long-term User / Agent / Task State

RAG
→ External Knowledge Retrieval

RAG 解决的是外部知识获取：



Memory 解决的是长期经验沉淀：



最核心区别是数据来源、生命周期和语义不同：

- RAG 的知识通常来自外部文档、网页、知识库、数据库
- Memory 的信息通常来自用户交互、Agent 行为、任务历史、工具观察
- RAG 的知识更像外部事实资源
- Memory 的信息更像 Agent 与用户共同积累的长期状态

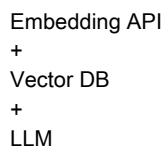
所以不要把 Memory 简化成“带用户信息的 RAG”。更准确地说：

Memory 和 RAG 都是 Runtime

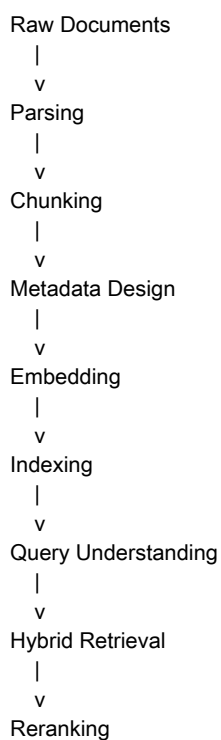
对“当前上下文之外的信息”的召回机制，但它们管理的信息类型和生命周期不同。

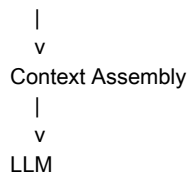
Knowledge Base 与企业 RAG

企业知识库不是：



更完整的 RAG Pipeline 是：





企业知识库的难点不是单点，而是整条链路。

文档解析

企业文档往往包含

PDF、Word、表格、图片、扫描件、页面结构、权限信息和版本关系。解析质量会直接影响后续 Chunk 和检索质量。

Chunking

Chunk 不是随便按长度切分。它要尽量保留语义完整性：

- 标题和正文的关系
- 表格和说明的关系
- 代码块和解释的关系
- 同一业务规则的完整边界

Metadata

Metadata 是企业知识库效果的关键部分。

例如：

```
{
  "doc_id": "policy_2026_08",
  "title": "Expense Policy",
  "section": "Travel Reimbursement",
  "department": "Finance",
  "permission": "employee",
  "version": "2026.08"
}
```

这些字段会影响过滤、排序、权限控制和解释来源。

Retrieval 与 Rerank

向量检索通常只是第一层候选召回。更成熟的系统还会结合 keyword、metadata、BM25、semantic search、reranker 和业务规则。

所以企业知识库的质量不只取决于“有没有向量数据库”，而取决于知识表示、切分策略、索引设计、召回策略、排序策略和 Context Assembly。

Memory Update 与 Similarity 的关系

用户提出了一个关键问题：

如果用户以前喜欢 Vue，后来喜欢 React，Memory 怎么知道这是同一个信息并进行更新？

答案是：不能只靠向量相似度。

工业级 Memory Update 往往需要：

Semantic Similarity
+

Structured Type
+
Entity / Slot Matching
+
Temporal Signal
+
LLM Judgment

例如：

Old Memory:
type = preference
slot = frontend_framework
content = User prefers Vue

New Candidate:
type = preference
slot = frontend_framework
content = User now prefers React

这两条 Memory 不只是语义接近，更是同一 type + slot 下的偏好变更。因此 Runtime 可以选择：

- update：把 Vue 更新为 React
- merge：保留“过去偏好 Vue，现在偏好 React”的历史
- create：如果新偏好只属于某个项目范围，则新建 project-scoped memory
- deprecate：把旧 Memory 标记为过期

Similarity 只提供“可能相关”的信号，不能替代生命周期决策。

Query Similarity 是动态的

单条 Memory 的向量通常是稳定的，但它和当前 Query 的 similarity 是动态的。

例如 Memory：

User prefers TypeScript.

当用户问：

我要写一个前端项目，技术栈怎么选？

它可能高度相关。

当用户问：

我要写一个 Java 后端服务。

它可能相关性很低。

所以：

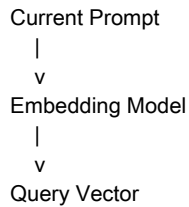
Memory Vector: relatively stable
Query Vector: changes every turn
Similarity: computed between current query and stored memory

这解释了为什么 Memory 不是永久带着一个固定 relevance 分数。relevance 永远是相对当前任务计算出来的。

Prompt 到 Embedding，不等于 Memory Extraction

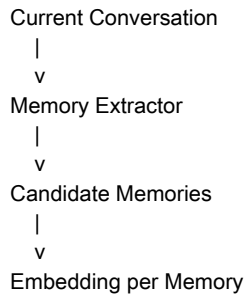
用户输入一段 Prompt 时，系统可能会做两类完全不同的事情。

第一类是 Query Embedding：



它用于当前这一轮的检索。

第二类是 Memory Extraction :



复杂 Prompt 可能被抽成多条 Memory :

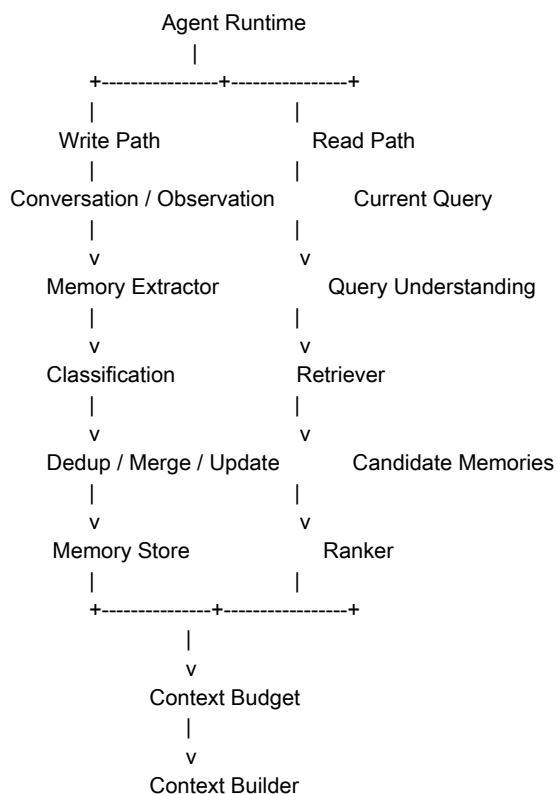
Prompt:
 "我最近在学 Agent Runtime，主要用 TypeScript，希望你以后解释时多讲原理。"

- Extracted Memories:
1. User is learning Agent Runtime.
 2. User mainly uses TypeScript.
 3. User prefers explanations with principles.

Query Embedding 是为了“现在找什么”，Memory Extraction 是为了“以后记住什么”。二者不能混为一谈。

Part B 最终模型

Part B 最终得到的 Memory Architecture 可以表示为：



|
v
LLM

这张图强调了两个核心事实：

- Memory 的写入和读取是两个不同问题
- Memory 最终必须通过 Context Builder 进入当前 LLM Context

本 Part 核心知识点

- Memory Architecture 由 Write Path 和 Read Path 组成
- Write Path 负责 extraction、classification、dedup、merge、update、forget、storage
- Read Path 负责 query understanding、retrieval、ranking、budget、projection
- Memory Store 是长期 State Repository，不是简单文本列表
- Memory Entity 需要 type、metadata、confidence、importance、time、scope、status
- Vector DB 是基础设施，不是 Memory System
- Embedding 是语义表示技术，不是 Memory 本身
- Retriever 负责召回候选，Ranker 负责排序与选择
- Hybrid Retrieval 通常结合 keyword、semantic、metadata、recency 等信号
- Context Budget 防止 Memory 污染当前任务
- Context Builder 把 Memory Record 投影成 LLM 可读上下文
- Memory 和 RAG 可共享 Retrieval 技术，但数据来源、生命周期和语义不同
- 企业知识库的难点在 parsing、chunking、metadata、retrieval、rerank、context assembly 整条链路
- Memory Update 不能只靠 similarity，需要结构化类型和生命周期判断
- Query similarity 是动态相关性，不是 Memory 自身固定分数

写书 TODO

Part B 建议沉淀以下章节：

1. Memory Architecture Overview

说明 Memory 为什么不是数据库问题，而是 Runtime 长期信息管理问题。

2. Memory Store and Memory Entity

展开 Memory Store 的领域抽象，以及 Memory Entity 的字段设计。

3. Embedding and Vector Search

解释 Embedding、Vector Search、Vector DB 的职责边界。

4. Retriever and Ranker

重点讲 Retriever 找候选，Ranker 排优先级，以及为什么相似度最高不一定最应该进入 Context。

5. Memory Projection into Context

承接 Day04 Context Builder , 说明 Memory 如何变成当前 LLM 可读的 Context Block。

6. Memory vs RAG

系统区分 Memory、RAG、Knowledge Base、Vector DB、Context Builder。

7. Enterprise Knowledge Base

单独整理企业知识库的 RAG

Pipeline : Parsing、Chunking、Metadata、Embedding、Retrieval、Rerank、Context Assembly。

写书素材

这一段可以作为书里的核心观点：

Vector DB 不是 Memory , Embedding 也不是 RAG。

完整 Agent Memory 是：

Memory
=
Extraction
+
Classification
+
Storage
+
Retrieval
+
Ranking
+
Lifecycle
+
Context Projection

企业知识库也不是：

Embedding API
+
Vector DB
+
LLM

真正决定知识库效果的是：

Knowledge Representation
+
Chunking
+
Metadata
+
Retrieval
+
Ranking
+
Context Engineering

本 Part 核心认知升级

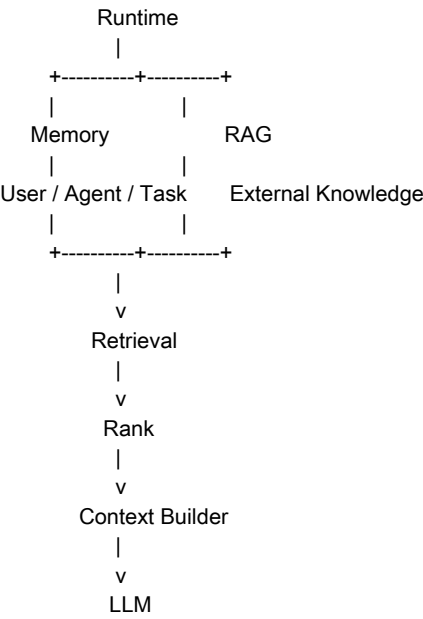
Part B 最大的认知升级是从：

Memory 就是把历史信息存下来。

升级到：

Memory 是 Runtime 对长期信息进行管理，并在当前任务需要时重新召回、排序和投影的机制。

进一步说：



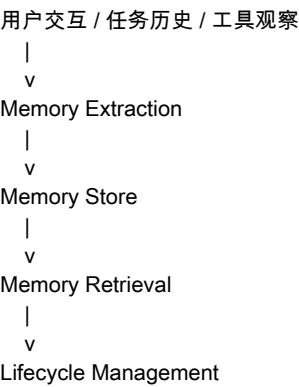
Memory 最终不是一个 Database，而是一种让 Agent 拥有“过去”的 Runtime 能力。

工业级实现

企业落地时，可以把系统拆成两个相邻但不同的子系统。

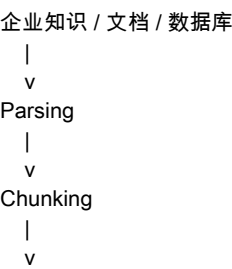
Agent Memory System

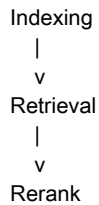
负责：



RAG / Knowledge System

负责：

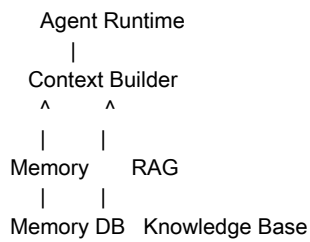




二者最后都进入：

Context Builder

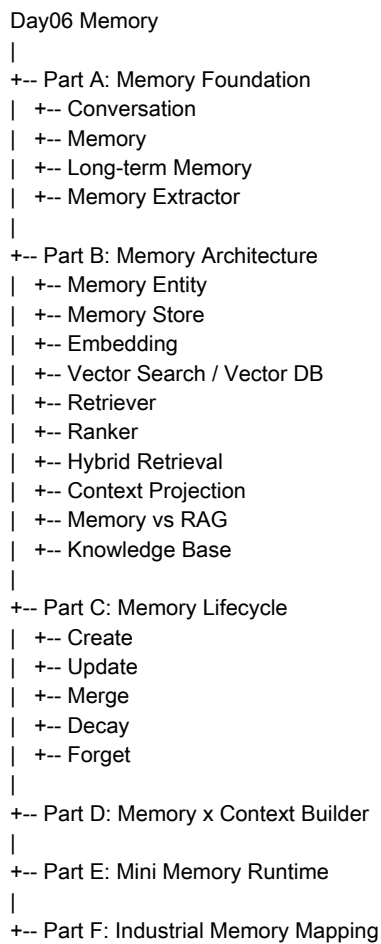
工业架构中更准确的说法是：



这比“我们的 Agent 有一个 Vector DB”准确得多。

知识地图

目前 Day06：



面试视角

如果面试官问：

Memory 和 RAG 有什么区别？

建议回答：

RAG 主要解决外部知识的按需检索，而 Memory 主要解决 Agent、User、Task 长期信息的持久化与召回。两者底层都可以使用 Embedding、Vector Search、Reranking 等 Retrieval 技术，但数据来源、生命周期和语义不同。

Vector DB 是不是 Memory？

建议回答：

不是。Vector DB 是 Memory Store 的一种基础设施实现。Memory 还包括信息提取、分类、存储、检索、排序以及生命周期管理。

Retriever 和 Ranker 有什么区别？

建议回答：

Retriever 负责从大规模 Memory Store 中召回候选，目标是找得全；Ranker 负责在候选中排序和筛选，目标是选得准。相似度只是 Ranking 信号之一。

企业 RAG 最核心的难点是什么？

建议回答：

不是简单地把文档向量化，而是如何把原始知识加工成适合检索的表示，并通过 Chunking、Metadata、Hybrid Retrieval、Reranking 和 Context Engineering，把真正相关的知识准确召回并提供给 LLM。

本章思考题

- 为什么 Memory Retrieval 不能把所有 Memory 都塞给 LLM？
- 为什么 Memory Vector 通常稳定，但 similarity 会随着 Query 改变？
- 为什么一个复杂 Prompt 可能应该提取成多个 Memory？
- 为什么 Vector DB 不等于 Memory？
- 为什么 RAG 可以没有 Memory？
- 为什么一个成熟企业知识库不能只做 Vector Search？
- 为什么 Chunking 会直接影响 Retrieval Quality？
- 为什么 Memory Update 不能单纯依靠 Vector Similarity？
- 为什么 Retriever 和 Ranker 应该分层设计？
- 为什么 Memory Injection 本质上是 Context Projection？

前置问题回收

Part B 已经解决：

- Memory 和 Conversation 的区别
- Memory 和 Vector DB 的区别

- Memory 和 RAG 的区别
- Query Vector 与 Memory Vector 的关系
- Retriever 与 Ranker 的职责边界
- Hybrid Retrieval 为什么必要
- Context Budget 为什么参与 Memory Injection
- Knowledge Base 与 Vector DB 的层级关系
- 企业知识库为什么不仅仅是 Vector Search
- Memory Update 为什么需要结构化信息和生命周期判断

还没有展开、留到后续：

- Memory 到底什么时候 Create
- Memory Update / Merge 怎么判断
- Vue 到 React 这种偏好冲突怎么处理
- Memory 什么时候 Forget
- Confidence / Importance / Recency 如何计算
- Memory Lifecycle 是 Rule 驱动还是 LLM 驱动
- Mini Memory Runtime 如何实现

下一节学习计划

下一节进入：

Day06 Part C : Memory Lifecycle

核心问题从“Memory System 怎么设计”转向：

Memory 是怎么活着的？

重点链路：

```

Create
|
v
Update
|
v
Merge
|
v
Decay
|
v
Forget

```

Part C 会重点解决：谁决定创建 Memory，什么情况下 Update、Merge、Forget，偏好冲突如何处理，以及 Confidence、Importance、Recency 如何参与生命周期。